anime-rag

Multimodal (Agentic) RAG over the Manga109-s manga dataset — individual NLP course project.

- **Code:** https://github.com/martin76ec/manga-multi-modal-rag
- **Built index (embeddings + metadata, no images):** https://huggingface.co/datasets/martin76ec/Manga109-s-indexes

Course project overview

Topic

RAG / Agentic RAG. Not built with LangChain/LangGraph — this is a deliberate, evidence-based deviation, not an oversight. While scoping this requirement, I prototyped the migration far enough to test it for real (not just install the packages):

- `google/gemma-4-26B-A4B-it`, the vision-capable model this project needs so the agent can judge a panel visually (not just from dialogue text), **does not reliably use native tool-calling** — confirmed with the raw HF Inference API directly, bypassing LangChain entirely: given a registered tool, it just answered in plain text claiming it had no tools.
- Wiring it into LangGraph's `create_react_agent` made this worse, not better: it **hallucinated a complete fake tool call** — a fabricated external URL and fictional panel data — instead of calling the real tool or reporting failure.
- Models that *do* tool-call reliably on the free-tier providers I have access to (`Llama-3.3-70B-Instruct`, `Qwen2.5-72B-Instruct`) are text-only. The one vision+tool-calling candidate I could find and reach (`Qwen2.5-VL-32B-Instruct`) also ignored the tool and hallucinated a text answer.

So: no model available to me does both vision and reliable native tool-calling at once. The hand-built agent in `agent.py` (a MRKL-style `Thought → Action → Observation → Final Answer` loop, prompted in plain text rather than via native function-calling, with retry/degeneration-detection logic added after hitting real provider-side failures — see Conclusions) is not a workaround for not knowing LangChain; it is the result of testing the LangChain path and finding it less reliable for this specific model. I'm glad to discuss the full investigation (evidence above) in the Q&A.

Problem and data

Problem: manga dialogue in Manga109-s is Japanese-only, and panels carry visual information (who's present, what they're holding, what's happening) that the transcribed dialogue often doesn't describe. The goal is to let someone who doesn't read Japanese ask a natural-language question in English and get an answer grounded in cited manga panels — using both the artwork and the dialogue, not just the text.

Data: Manga109-s (gated, hal-utokyo) — 87 commercially-licensed manga volumes with page images and per-panel annotations (frame boxes, face/body boxes, transcribed dialogue text, character names). Fully indexed for this project: **87 books, 82,124 panels**.

Baseline vs. proposed solution

	Baseline	Proposed
Retrieval	single modality: text-only **or** image-only CLIP cosine similarity	blended multimodal score (`0.6 x image + 0.4 x text` , tunable) over the same CLIP embeddings
Query language	Japanese only (raw CLIP tokenizer)	English or Japanese (NLLB-200 translation bridge both directions)
Answering	top-k panels returned directly, no reasoning	MRKL+IRCoT agent: multi-round retrieval driven by the model's own reasoning, vision-grounded (sees the actual panel images, not just translated captions), cites book/page, explicitly says "I don't know" instead of guessing when evidence is insufficient

Comparison methodology and results

No formal labeled benchmark (hit-rate@k over a random query sample, or RAGAS) was built for this submission — this is a stated limitation, not an oversight (see Conclusions). What follows is a grounded comparison from real, development-time test queries, each one manually verified against the actual panel image, not just trusted at face value:

Modality ablation (same query, `"police officer"` , isolating each embedding):

Mode	Top-1 result	Score	Verified correct?
Image-only (`w_img=1.0, w_text=0.0`)	`AisazuNihaIrarenai_007`	0.256	not independently verified — no associated dialogue text, visibly weaker signal
Text-only (`w_img=0.0, w_text=1.0`)	`AisazuNihaIrarenai_057`	0.740	**yes** — panel literally reads "ちょっと警察の方!!" ("Wait... officer!!")
Blended (default `0.6/0.4`)	`AisazuNihaIrarenai_057`	0.403 (= `0.6x0.178 + 0.4x0.740` , checked by hand)	**yes**, same correct panel, arithmetically consistent

Isolating either modality changes the ranking; for this query text carried more signal, but the blend is what a general-purpose default needs to be robust across queries where the opposite is true (e.g. a purely visual query like "someone holding a gun," see below).

Baseline retrieval vs. agent (development-time test queries, qualitative "task resolved" read):

Query	Baseline (direct search)	Agent (MRKL+IRCoT)
"police officer"	correct top-1, single shot	same, 1 round
"What happens when a police officer shows up at someone's house?"	would need the user to already know to search for "police officer"	correctly finds and cites the same panel unprompted, synthesizes a grounded answer about the missing-son plot thread
"someone holding a gun" (only 2 books indexed, neither containing a real gun panel)	returns plausible-looking but wrong top results (no gun actually visible)	**correctly reports it cannot find a match**, instead of presenting a wrong result as right
"someone holding a gun" (after indexing `DollGun`, a real match exists)	finds the correct panel, no explanation	finds and **visually confirms** the panel ("a close-up of a Glock handgun"), a detail the dialogue text doesn't mention
"The police visit a house about a missing son. What is the father's name, and what does he say about being a parent?"	can't answer multi-hop questions at all (one query, one ranked list)	ran 4 distinct retrieval rounds with self-generated follow-up queries; correctly found the father's identity, then **honestly admitted** it couldn't confirm his exact words on parenting rather than guessing (the matching line existed but was mistranslated past recognition — a translation-quality limit, not a reasoning failure)

Reading on this: the agent's main measurable advantage over the baseline is correctly saying "I don't know" instead of confidently returning a wrong top-1 when evidence genuinely isn't there, and chaining multiple rounds of self-generated queries for questions the user couldn't have phrased as a single good search term. Its cost (next section) is substantial, so it is not a strict upgrade — it's a different point on the precision/cost trade-off, appropriate when a question can't be answered in one retrieval.

Cost comparison

	Baseline (direct search)	Proposed (agent)
Where it runs	fully local (CPU/ GPU)	retrieval local; reasoning calls the HF Inference API (network-dependent, third-party provider)
LLM calls per question	0	1 per round, up to `max_steps=4` (+1 forced final) = up to 5
Tokens per call	0	`max_tokens=250` output, plus the full conversation history (all prior Observations) re-sent each round
Images sent to the model	0	up to `top_k=3` panels/round x up to 4 rounds, resized to 384px/JPEG q80 before sending
Observed latency	sub-second	tens of seconds for a multi-round question
Monetary/account cost	none	HF Inference Provider usage (free tier used here; hit a 429 rate limit once during testing)

Local compute/storage (one-time, both modes share this):

	Value
Panels indexed	82,124 (87 books)
Embedding throughput	~58 panels/sec (shared/contended RTX 4060 Ti, 8GB)
Local models	`line-corporation/clip-japanese-base` (~790MB), `facebook/nllb-200-distilled-600M` (~2.4GB) — both free, local, no API key
Built index size	380.5MB (single Parquet file, embeddings + metadata, no images)
Raw data on disk (images + crops, full dataset)	~7.8GB

Conclusions

What worked:

- Blending image and text embeddings genuinely changes retrieval outcomes versus either alone — verified by hand, not assumed (see ablation table).
- The translation bridge (NLLB-200, both directions) makes the system usable by someone who doesn't read Japanese, at both query and result-display time, with no API key needed (fully local).
- The agent's vision pathway is not cosmetic: it found and correctly described a panel's content ("a Glock handgun") that the dialogue text never mentioned.
- The agent's abstention behavior ("I cannot find this") held up repeatedly instead of hallucinating a plausible-looking wrong answer, which is the behavior that actually matters for a citation-grounded tool.

What didn't work, and had to be fixed along the way (kept here deliberately — this is what "what worked / what didn't" is supposed to surface):

- CLIP collapsed very short/punctuation-only dialogue (e.g. "……") toward a generic embedding, polluting rankings with spurious high scores — fixed with a minimum meaningful-text filter.
- NLLB dropped key words (lost "police" entirely) when translating stuttering manga dialogue ("ちょっ...ちょっと") — fixed by adding beam search and repetition controls to generation.
- The HF Inference Provider occasionally returned malformed output: once an empty response disguised as a tool-call, twice two different repetition-loop patterns (plain word repetition, and a templated-but-varying repeated string that word-frequency detection initially missed). All required actual reproduction and root-causing, not just a retry wrapped around the symptom.
- The LangChain/LangGraph migration attempt (see Topic, above) failed outright for this model — a real negative result, not a skipped step.

Limits of this experiment:

- No formal quantitative benchmark (hit-rate@k over a representative, non-cherry-picked query sample, or RAGAS-style scoring) was built; the comparison above is real but small-N and development-time-selected, not a random sample.
- Retrieval **recall** is the dominant failure mode, not precision: a correct answer that isn't in the indexed subset simply won't be found, and the agent can only correctly say so, not conjure it. This showed up directly when only 2 of 87 books were indexed.
- Character-name translation is unreliable (NLLB has no manga-specific name dictionary), which can make an agent's cited quote technically correct in meaning but wrong in the exact attributed name.
- The agent's reliability depends partly on third-party HF Inference Provider behavior outside this project's control (observed transient malformed/degenerate outputs from one provider).

AI-generated content declaration

This project was built with extensive assistance from Claude (Anthropic), used as a pair-programming/ engineering collaborator throughout: architecture decisions, implementation, debugging real failures (translation collapse, embedding collapse, provider-side degenerate output, the LangChain/LangGraph investigation above), and this documentation. Design decisions, testing direction, and acceptance of each change were mine; the AI tool did the hands-on implementation and investigation under that direction, and its claims were repeatedly verified against actual run output and real data rather than taken at face value (e.g. every "it works now" claim in the project history was backed by a reproduced test, not just stated). Adjust this section's wording as needed for your institution's exact disclosure policy before submitting.

Presentation (5 min + 3 min Q&A)

Suggested structure for the 5-minute walkthrough: problem/data (30s) -> topic + the LangChain/ LangGraph finding (45s) -> live demo of `anime-rag ask` (text query, then an agent query that needs multi-hop reasoning) (2 min) -> comparison table + cost table (1 min) -> conclusions/limits (45s). The

Q&A is expected to probe code and decisions directly — the sections above are written to be defensible point-by-point, not just descriptive.

Setup

```
uv sync
hf auth login                # needed once, for the gated dataset
```

Manga109-s is a gated dataset — request access on the dataset page first.

Usage

```
# list the 87 available books
uv run anime-rag books

# extract + embed a few books (fetches only those books from the dataset zip,
# via HTTP range reads — no need to download the full ~3GB archive)
uv run anime-rag index AisazuNihaIrarenai ARMS

# search in English — the query is auto-translated to Japanese before
# embedding (the CLIP model and the dialogue are both Japanese-native), and
# each result's dialogue is translated back to English for display
uv run anime-rag search "police officer"

# search in Japanese directly (skips translation, since it's detected)
uv run anime-rag search "警察官"

# search by image — finds visually similar panels
uv run anime-rag search --image path/to/some_panel.jpg

# tune the image/text score blend, save result crops for inspection
uv run anime-rag search "missing son" --w-img 0.4 --w-text 0.6 --save-crops out/

# skip translation entirely (faster startup if you're querying in Japanese)
uv run anime-rag search "警察官" --no-translate

# interactive REPL: loads models once, then take as many questions as you like.
# menu-driven (search by text / by image / ask the agent / quit), colored
# output, and an inline true-color thumbnail of the top result panel
uv run anime-rag ask

# agentic Q&A: a MRKL+IRCoT agent that issues its own retrieval queries over
# several rounds and synthesizes a cited answer (needs HF Inference access)
uv run anime-rag agent "What happens when a police officer shows up at someone's
        house?"
```

How it works

- `source.py` opens the dataset's zip archive through `HfFileSystem` and extracts only the requested books' images/annotations (partial HTTP range reads).
- `annotations.py` parses each book's Manga109 XML into pages of frames (panels), text boxes, and character face/body boxes.
- `panels.py` crops one image per frame and associates it with the dialogue text and character names whose boxes fall inside that frame.
- `embedder.py` wraps the Japanese CLIP model to embed panel crops and text into a shared space.
- `translate.py` translates English queries to Japanese before embedding, and Japanese result dialogue back to English for display, via `facebook/nllb-200-distilled-600M` (local, no API key needed).
- `indexing.py` builds and persists a per-book index (embeddings + metadata) under `data/index/`.
- `search.py` ranks panels for a query by a weighted blend of image-embedding and text-embedding cosine similarity. Panels whose dialogue is empty or too short to embed meaningfully (e.g. "……") fall back to image-only scoring.
- `agent.py` implements a **MRKL + IRCoT** retrieval agent on top of `search.py`: a MRKL-style explicit `Thought → Action → Observation → Final Answer` transcript (`llm.py`, via the HF Inference API — the only part of this project that's not local) drives a single `search_manga` tool, but IRCoT-style, each retrieval query is the agent's own latest reasoning step rather than a fixed query, so it can chase multi-hop questions across several retrieval rounds and still ground its answer in cited book/page evidence (or admit it doesn't know, instead of guessing). The LLM (`google/gemma-4-26B-A4B-it`) is vision-capable, so each Observation includes the actual panel images, not just translated dialogue — it can judge what's visually happening in a panel even when the dialogue doesn't describe it.

Downloaded data and built indices live under `data/` (gitignored) — the dataset license forbids redistribution.

`anime-rag agent` calls the Hugging Face Inference API (model: `google/gemma-4-26B-A4B-it` by default) and needs network access plus an HF token with inference access (`hf auth login`); everything else in this project (embedding, translation, indexing) runs fully locally.

Indexing the full dataset (e.g. on a bigger GPU)

The dataset has 87 books (82,124 panels total once all are indexed). Embedding that locally on a small/shared GPU takes a while, so on a bigger box (this was written with an H200 in mind):

```
# on the bigger machine: clone this repo, then
uv sync
hf auth login                       # needs Manga109-s access

# index every book in one go; bump --batch-size way up on a big GPU
uv run anime-rag index --all --batch-size 256

# or, as a single script (also pushes the result to HF when given a repo id):
./scripts/h200_index_all.sh yourname/manga109-rag-index
```

`--all` opens the dataset zip once and indexes everything it finds via `anime-rag books`, skipping any book already indexed (safe to re-run / resume after an interruption).

Moving the index to another machine

`push-index` / `pull-index` upload/download **only the built index** (embeddings + metadata) via a HF dataset repo — never the actual panel crop images, which live in a separate directory (`data/raw/crops/`) the license is stricter about. This is derived numeric output, analogous to publishing a pretrained model, not a copy of the dataset itself — see the license note below. Push it **private** (this project's own index, linked above, is public by request).

The index is consolidated into a single `index.parquet` file before uploading — Parquet is the standard single-file format for embeddings + metadata (what HF's own `datasets` library uses), and far easier to move around than one directory per book (87 books = 348 small files otherwise). `pull-index` downloads that one file and transparently re-splits it back into the usual `data/index/<book>/` layout, so nothing else changes.

```
# on the box that built the index
uv run anime-rag push-index yourname/manga109-rag-index

# if you previously pushed with an older version of this tool that uploaded
# the raw per-book folders instead of a single file, add --clean once to
# remove that old structure from the repo after the parquet file is up
uv run anime-rag push-index yourname/manga109-rag-index --clean

# on another machine
uv run anime-rag pull-index yourname/manga109-rag-index

# the pulled index has no crop images yet -- regenerate them locally
# (re-extracts raw pages from the dataset zip, crops them; no GPU/model
# inference involved, so it's cheap even without the bigger box)
uv run anime-rag crops
```

Search/citations work immediately after `pull-index` (it's pure embedding math); you only need `anime-rag crops` once you want to actually view a result's panel image or use the vision-capable agent.

License note

Manga109-s is distributed under its own license (non-commercial-friendly but not open), which caps how much of any one book's pages may be published and forbids redistributing the dataset itself. See the dataset page for details.